

Roles Module - Guide Manual

Role Management: system roles, custom roles, hybrids and permission keys

Generated 5 September 2026 · Source: docs/ · Internal documentation

ATSS 2.0 · Sidebar → **Users** → **Roles** (Role Management)

This manual covers the Role Management screen: the fields on the role form, what a permission key means, how a role built on a system role behaves, and the rules the server enforces when you save.

1. What a role is

A role is a named set of **permission keys**. A key either opens a page (`job-order`) or enables one control on that page (`job-order.approve`). A user holds exactly one role, on `users.role_id`.

There are two kinds:

Kind	IDs	Editable here?	Where its access comes from
System role (seeded, "locked")	1-8	No — shown with a <i>System</i> badge and a <i>Locked</i> label	<code>backend/app/Support/Permissions.php</code> → <code>ROLE_PERMISSIONS</code>
Custom role	9 and above	Yes — Edit and Delete	its own stored <code>permissions</code> list, plus its base role if it has one

The eight system roles

ID	Role	Lands on after sign-in
7	SuperAdmin	<code>dashboard</code>
1	Administrator	<code>dashboard</code>
8	Head Technician	<code>application-management</code>
2	Technician	<code>job-order</code>
6	OSP	<code>work-order</code>
5	Inventory Staff	<code>inventory</code>
4	Agent	<code>agent-dashboard</code>
3	Customer	<code>customer-dashboard</code>

SuperAdmin holds the **wildcard** `*` — every key, including keys added later.

The server is always the authority. Every endpoint is checked against `backend/app/Support/ApiPermissionMap.php` whatever the browser drew. `frontend/src/config/permissions.ts` is only the client's copy, kept in step so the UI can decide what to *draw* without a round trip. **Change the backend file first.**

2. Screen — Role Management (`frontend/src/pages/roles.tsx`)

List columns

Column	Shows
Role Name	the name, with a System badge for IDs 1–8 and a purple <code><Base> +</code> badge for a hybrid
Description	<code>description</code> , or "No description provided"
Last Updated	<code>updated_at</code> , date only
Actions	Edit / Delete for a custom role; the word Locked for a system role

Search filters on role name. Page size: 10 / 25 / 50 / 100.

Controls and the keys they need

Control	Permission key
Open the page	<code>roles</code>
+ (Add)	<code>roles.create</code>
Edit (pencil)	<code>roles.edit</code>
Delete (trash)	<code>roles.delete</code>

A control is only drawn when the request behind it would actually succeed.

Organization scoping

The list shows system roles ($ID \leq 8$) **plus** roles belonging to your organization. A custom role from another organization is filtered out on the client and refused by the server (403) on update and delete.

3. Fields — Add / Edit Role modal (`modals/RoleModal.tsx`)

Field	Required	Type	Stored as	Rules
Role Name	Yes	text, max 255	<code>roles.role_name</code>	Must be unique across the whole table , system roles included. A duplicate returns 422.

Field	Required	Type	Stored as	Rules
Description	No	textarea (2 rows)	<code>roles.description</code>	Free text. Shown in the list.
Start From a System Role	No	select	<code>roles.base_role_id</code>	One of the eight system roles, or "None — <i>pick every page by hand</i> " (stored as <code>NULL</code>). Nothing else is accepted — a custom role can never be a base, so roles can never chain.
Permissions	No	checkbox grid	<code>roles.permissions</code> (JSON array)	Only the keys ticked here are stored — see §5.
Organization	auto	—	<code>roles.organization_id</code>	Taken from the signed-in user on create. Cannot be changed on update — the API strips it.
Created / Updated by	auto	—	<code>roles.created_by_user_id</code> , <code>updated_by_user_id</code>	The signed-in user.
Permissions version	auto	—	<code>roles.permissions_version</code>	Stamped to <code>Permissions::CURRENT_VERSION (1)</code> on every save. Clients cannot set it. See §7.

The permission grid

Three columns: **Page Name** • **View** • **Actions**.

- **View** opens the page — it is the page key itself (`job-order`).
- Each **Action** beside it is one button on that page (`job-order.approve`). Leave an action unticked and that button is hidden for the role.
- Rows are grouped under the same section headings the sidebar uses, so a role is ticked in the shape it will be navigated in.
- The grid is generated from the permission catalog, so a page added to the catalog becomes grantable immediately — it never has to be listed twice.

Behaviour while ticking

You do	What happens
Tick an action	Its page (View) is ticked automatically — unless the base role already grants it.
Untick a page (View)	Every action under it is unticked.
Tick one of an exclusive pair	The partner is cleared (see below).
Choose / change a base role	Every extra the new base already grants is dropped (it would be a stale duplicate), as is anything exclusive with what the base grants.

The mutually exclusive pairs

These cannot both be held — one opens the technician's Done form, the other the administrator's:

- `job-order.tech-edit` ↔ `job-order.admin-edit`
- `service-order.tech-edit` ↔ `service-order.admin-edit`

If a **base role** brings one of a pair in, the other is not offered at all — a hybrid cannot un-inherit half its base.

4. Hybrid roles — "Start From a System Role"

Pick one of the eight and the new role **holds everything that role holds**, then the ticks below only **add** to it.

- Inherited keys appear **ticked, locked, and badged *Inherited***, with a tooltip naming the base.
- The inheritance is **live**: it is resolved from `Permissions` on every read, never copied into the role's stored list. So the hybrid follows its base role as that role changes, including pages added to it later.
- Choosing **SuperAdmin** as the base inherits the wildcard — the modal says so, and there is nothing left to add; the stored `permissions` are saved empty.
- A **system role is never itself a hybrid**. Its access is the table in `Permissions`, so a `base_role_id` recorded against IDs 1–8 is ignored rather than quietly widening a seeded role.
- Clearing the base back to *None* turns the role standalone. Its own ticked keys are untouched — it simply loses the inherited half.

5. What is stored versus what is granted

effective access = base role's keys (live) + this role's stored permissions

Only the **extras** are written to `roles.permissions`. Storing the inherited half would freeze a copy of the base role at the moment of saving, which is the one thing hybrids exist to avoid.

The API returns two fields on a role:

Field	Meaning
<code>permissions</code>	the column exactly as stored
<code>effective_permissions</code>	what the role actually grants, grandfathering included, minus the inherited half

The modal seeds its checkboxes from `effective_permissions`. This matters: a role saved before per-action keys existed stores only its pages, so seeding from the raw column would show Add / Edit / Delete unticked for a role that has them — and the first save would then silently revoke them from a screen that never showed them ticked.

6. Permission key reference

Naming

- Page key: the page id — `job-order`, `customer`, `agent-payout`.

- Action key: `<page>.<verb>`.
- Standard verbs (`Permissions::CRUD_VERBS`, shown in this order): `create` → **Add**, `edit` → **Edit**, `delete` → **Delete**.
- Anything that is not one of the three keeps a descriptive verb — `job-order.approve` says what it grants; `job-order.action3` would not.

The groups shown in the grid

Group	Pages
Dashboards	dashboard, agent-dashboard, live-monitor, support
Billing	customer, transaction-list, transactions-revert, payment-portal, soa, invoice, overdue, so-charge, dc-notice, mass-rebate, staggered-payment, discounts, soa-generation
Operations	application-management, job-order, service-order, radius-queue, work-order, lcp-nap-location, sms-blast, reports
Agent	bonus-history, agent-invoices, agent-payout, agent-management, team-agent
Inventory	inventory, inventory-category-list
Tools	smartolt-tool, mikrotik-radius-tool, xendit-reconcile-tool, billing-reconcile-tool
Configurations	promo-list, plan-list, location-list, lcp, nap, ports, router-models, status-remarks-list, usage-type, vlan-config, payment-method, work-category, radius-config, smart-olt, sms-config, sms-template, email-templates, pppoe-setup, concern-config, billing-config
Users	user-management, tech-users, organization, roles, group-management
Logs	disconnected-logs, reconnection-logs, sms-logs, sms-blast-logs, email-logs, data-logs, expenses-log, smart-olt-logs, radius-logs, system-logs
Customer Portal	customer-dashboard, customer-bills, customer-support, agent-application
Settings	settings
Other	any page not filed above — appended automatically so nothing is ungrantable

Non-standard action keys

Page	Actions
job-order	<code>approve</code> , <code>failed</code> , <code>tech-edit</code> , <code>admin-edit</code> , <code>attachment</code> , <code>pre-install</code>
customer	<code>so-request</code> , <code>details-edit</code> , <code>attachment</code> , <code>transact</code>
transaction-list	<code>batch-approve</code> , <code>approve</code> , <code>revert-request</code>
application-management	<code>move-to-jo</code> , <code>quick-status</code>
service-order	<code>tech-edit</code> , <code>admin-edit</code>

Page	Actions
work-order	<code>manage</code> (raise / reassign / delete, as opposed to working one assigned to you)
reports	<code>manage</code> , <code>delete</code>
bonus-history	<code>payout</code>
agent-payout	<code>approve</code>
agent-invoices	<code>generate</code> , <code>status</code> , <code>payout</code>
soa-generation	<code>manage</code>
mass-rebate / staggered-payment / discounts	<code>add</code>

Every **Configurations** page and every **Users** page uses the standard `create` / `edit` / `delete` trio.

Agent Management currently has no per-action keys of its own. It renders the User Management screen with `agentOnly` , and its Add / Edit / Delete controls are ungated — holding the `agent-management` page grants all three, until that group is given its own verbs.

7. Grandfathering and retired keys

Two compatibility rules exist so that splitting page keys into per-action keys did not silently revoke buttons from roles saved earlier.

Grandfathered actions — a role stored under an older `permissions_version` that holds one of these pages is also granted its standard verbs: every Configurations page, plus `tech-users` , `organization` , `roles` , `group-management` ; `status-remarks-list` → `create` only; `user-management` → `create` and `delete` only. **Saving the role from this modal stamps the current version and the ticks become authoritative** — that is the only way a role leaves the rule.

Retired keys — still read, no longer offered: `ports.manage` , `router-models.manage` , `status-remarks-list.manage` . Each grants all three of its page's standard verbs. Re-saving the role writes the new keys.

8. Rules the server enforces

Rule	Response
System roles (ID ≤ 8) cannot be edited	403 "System roles cannot be edited"
System roles cannot be deleted	403 "System roles cannot be deleted"
A role with users assigned cannot be deleted	400 "Cannot delete role that has assigned users" — reassign those users first

Rule	Response
<code>role_name</code> must be unique	422
<code>base_role_id</code> must be one of IDs 1-8, or null	422
Every permission key must be one the system recognises	422 — an unknown key would silently grant nothing, which reads as a broken permission system rather than as the typo it is
A role from another organization	403 on update and delete
<code>organization_id</code> and <code>permissions_version</code> in the payload	ignored on update

Failed creates and updates are also written to the Laravel log (`Role create failed` / `Role update failed`) with the name and base role.

9. Assigning a role to a user

Roles are not assigned from this screen. Go to **Users → User Management**, open the user, and pick the role in the **Role** field (`users.role_id`).

- **Agent Management** locks that field to **Agent** and reveals the agent terms — see the Agent Module manual.
- A role change takes effect on the user's **next request**: the server re-reads their role, so an administrator whose role is changed mid-session loses the access immediately rather than at their next sign-in.
- At sign-in the API returns the user's `role`, `role_id`, resolved `permissions` and the page they should land on, so a client never has to work out which kind of role it holds.

10. Data model reference

roles

Column	Type	Notes
<code>id</code>	bigint PK	1-8 are the seeded system roles
<code>organization_id</code>	bigint, nullable	
<code>role_name</code>	string	UNIQUE
<code>description</code>	text, nullable	
<code>base_role_id</code>	int, nullable	the seeded role a hybrid builds on
<code>permissions</code>	longText, cast to array	the role's own keys only

Column	Type	Notes
<code>permissions_version</code>	int	generation of the permission model this row was saved under
<code>created_by_user_id</code> , <code>updated_by_user_id</code>	bigint, nullable	
<code>created_at</code> , <code>updated_at</code>	timestamps	

`user_roles` — `user_id`, `role_id`, `organization_id`, `timestamps`. The primary assignment used everywhere is `users.role_id`.

11. API reference

Method	Endpoint	Notes
GET	<code>/api/roles</code>	list, with <code>users_count</code> and <code>effective_permissions</code>
POST	<code>/api/roles</code>	create
GET	<code>/api/roles/{id}</code>	one role, with its users
PUT	<code>/api/roles/{id}</code>	update (refused for ID ≤ 8)
DELETE	<code>/api/roles/{id}</code>	delete (refused for ID ≤ 8, or when users are assigned)
POST	<code>/api/users/{id}/roles</code> · DELETE <code>/api/users/{id}/roles</code>	assign / remove

Create and update payload:

```
{
  "role_name": "Billing Supervisor",
  "description": "Billing pages plus transaction approval",
  "base_role_id": 1,
  "permissions": ["transaction-list.approve", "reports"],
  "organization_id": 4
}
```

`permissions` carries **extras only** — never the inherited keys, which the server merges in on every read.

12. Troubleshooting

Symptom	What to check first
---------	---------------------

A page is missing from the sidebar for a role	The page key is not held. Sidebar visibility is decided by the permission table, not by the menu.
A button is missing but the page opens	The page key is held and the action key is not.
A ticked box is greyed out	It is inherited from the base role (badged <i>Inherited</i>), or its exclusive partner is inherited.
Add / Edit / Delete vanished from a Configurations page after saving a role	The role was grandfathered; saving stamped the current version and made the ticks authoritative. Tick the three verbs explicitly.
Both Done forms wanted on one role	Not possible — <code>tech-edit</code> and <code>admin-edit</code> are mutually exclusive by design.
"Cannot delete role that has assigned users"	Move those users to another role first.
Role name rejected as taken	Names are unique across the whole table, including the eight system roles and roles from other organizations.
A new page cannot be granted to anyone	Add it to <code>Permissions::PAGES</code> (backend, first) and to <code>PAGES / PERMISSION_LABELS</code> in <code>config/permissions.ts</code> , then map its endpoints in <code>ApiPermissionMap.php</code> .